UQÀM — Département d'informatique

MGL7230 : Tests Logiciels

Hiver 2026

Devoir 2 – Rapport Teste Unitaire JUnit

Préparé par :

Assane LO

Code permanent : LOXA63360006

Courriel : lo.assane.2@courrier.uqam.ca

Remis au : **Pr Jameleddine Hassine**

Date de remise : le 6 mars 2026

1. Suite de tests JUnit

1.1 Structure du projet de tests

Le projet est organisé suivant l'architecture Maven standard avec une séparation claire entre le code source et les tests. La structure respecte les conventions de nommage et d'organisation de JUnit 5.

La structure des tests reflète fidèlement l'organisation du code source, suivant le principe de séparation par couches (domaine, repository, service). Chaque classe de test est nommée selon la convention <ClasseSousTest>Test.java, ce qui permet au plugin Maven Surefire de les détecter automatiquement.

Le projet utilise uniquement JUnit Jupiter (JUnit 5) avec le scope test, garantissant que les bibliothèques de test ne sont pas incluses dans l'artefact final. L'annotation @DisplayName est utilisée systématiquement pour améliorer la lisibilité des rapports de tests.

1.2 Aperçu des cas de test

Le projet contient **11 tests unitaires** répartis sur **3 classes de test**, conçus pour détecter des bugs intentionnellement introduits dans le code source. Chaque test cible un comportement spécifique ou une condition limite susceptible de révéler une défaillance.

Le tableau suivant résume les cas de test développés par classe :

Classe TimeSlotTest (3 tests)

Test	Objectif	Assertion clé
testTouchingEndpointsNoOverlap	*Vérifier que des créneaux adjacents (ex: [9:00-10:30) et [10:30-12:00)) ne sont **pas** considérés comme se chevauchant*	*assertFalse(slot1.overlapsWith(slot2))*
testActualOverlap	*Vérifier la détection correcte d'un chevauchement réel (ex: [9:00-10:30) et [10:00-11:30))*	*assertTrue(slot1.overlapsWith(slot2))*
testNoOverlapDifferentDays	*Vérifier que des créneaux sur des jours différents ne*	*assertFalse(monday.overlapsWith(tuesday))*

	se chevauchent jamais	

Stratégie de test : Tests de valeurs limites (boundary testing) sur les points de jonction entre créneaux.

CourseCatalogTest (4 tests)

Cette classe teste la robustesse du catalogue de cours face aux variations de saisie utilisateur (casse, espaces).

Test	Objectif	Assertion clé
testGetCourseNormalizedCase	*Vérifier que la recherche de cours est insensible à la casse ("inf101", "Inf101", "INF101")*	*assertNotNull(catalog.get("inf101"))*
testGetCourseWithWhitespace	*Vérifier que les espaces avant/après sont ignorés (" INF101 ")*	*assertNotNull(catalog.get(" INF101 "))*
testGetNonExistentCourse	*Vérifier le comportement pour un cours inexistant*	*assertNull(catalog.get("INF999"))*
testAddAndSize	*Vérifier l'ajout de cours et le comptage correct*	*assertEquals(2, catalog.size())*

Stratégie de test : Tests de robustesse (robustness testing) et tests négatifs.

EnrollmentServiceTest (4 tests)

Test	Objectif	Assertion clé
testTouchingTimeSlotsNoConflict	*Inscrire un étudiant à deux cours adjacents (devrait réussir)*	*assertNotNull(enroll1) et assertNotNull(enroll2)*
testScheduleConflict	*Vérifier le rejet d'inscription en cas de vrai chevauchement*	*assertThrows(EnrollmentException.class)*

testCourseNotFoundNormalization	*Inscrire avec code en minuscules (devrait réussir si normalisé)*	*assertNotNull(enrollment)*
testCapacityIncrementAfterValidation	*Vérifier que la capacité n'augmente pas si inscription échoue*	*assertEquals(0, course.enrolledCount())*

Stratégie de test : Tests d'intégration entre composants, tests de flux de validation (transaction testing).

Techniques d'assertion utilisées :
assertNotNull() / assertNull() : Validation de présence/absence
assertTrue() / assertFalse() : Validation booléenne
assertEquals() : Validation de valeur exacte
assertThrows() : Validation d'exceptions attendues

Méthodes de préparation :
Utilisation systématique de @BeforeEach pour initialiser un contexte propre avant chaque test, garantissant l'isolation et la répétabilité des tests (principe FIRST : Fast, Isolated, Repeatable, Self-validating, Timely).

Messages personnalisés :
Tous les tests incluent des messages descriptifs dans les assertions pour faciliter le diagnostic en cas d'échec (ex: "get() devrait normaliser en majuscules").

Les fichiers sources des tests sont fournis dans l'archive HW2-tests.zip jointe au devoir.

2. Exécution des tests — Présence de bugs

Cette section présente les résultats de l'exécution de la suite de tests sur le code original (non corrigé). Plusieurs tests échouent, ce qui révèle la présence des bugs intentionnels.

2.1 Capture d'écran des résultats

```
Windows PowerShell
PS C:\Users\assan\Desktop\Workspace\mgl7230\Devoirs\Devoir2\junit-homework2\junit-homework2> mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< edu.latece:junit-homework >-----------------------
[INFO] Building JUnit Homework Project 1.0.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ junit-homework ---
[INFO] skip non existing resourceDirectory C:\Users\assan\Desktop\Workspace\mgl7230\Devoirs\Devoir2\junit-homework2\junit-homework2\src\main\resources
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ junit-homework ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.4.0:testResources (default-testResources) @ junit-homework ---
[INFO] skip non existing resourceDirectory C:\Users\assan\Desktop\Workspace\mgl7230\Devoirs\Devoir2\junit-homework2\junit-homework2\src\test\resources
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ junit-homework ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ junit-homework ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running edu.latece.junit.domain.TimeSlotTest
[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 0.111 s <<< FAILURE! -- in edu.latece.junit.domain.TimeSlotTest
[ERROR] edu.latece.junit.domain.TimeSlotTest.testTouchingEndpointsNoOverlap -- Time elapsed: 0.016 s <<< FAILURE!
org.opentest4j.AssertionFailedError: Créneaux adjacents (fin de slot1 = début de slot2) ne devraient PAS se chevaucher ==> expected: <false> but was: <true>
        at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
        at org.junit.jupiter.api.AssertionFailureBuilder.buildAndThrow(AssertionFailureBuilder.java:132)
        at org.junit.jupiter.api.AssertFalse.failNotFalse(AssertFalse.java:63)
        at org.junit.jupiter.api.AssertFalse.assertFalse(AssertFalse.java:36)
        at org.junit.jupiter.api.Assertions.assertFalse(Assertions.java:239)
        at edu.latece.junit.domain.TimeSlotTest.testTouchingEndpointsNoOverlap(TimeSlotTest.java:18)
        at java.base/java.lang.reflect.Method.invoke(Method.java:580)
        at java.base/java.util.ArrayList.forEach(ArrayList.java:1596)
        at java.base/java.util.ArrayList.forEach(ArrayList.java:1596)

[INFO] Running edu.latece.junit.repo.CourseCatalogTest
[ERROR] Tests run: 4, Failures: 2, Errors: 0, Skipped: 0, Time elapsed: 0.054 s <<< FAILURE! -- in edu.latece.junit.repo.CourseCatalogTest
[ERROR] edu.latece.junit.repo.CourseCatalogTest.testGetCourseNormalizedCase -- Time elapsed: 0.015 s <<< FAILURE!
org.opentest4j.AssertionFailedError: get() devrait normaliser en majuscules ==> expected: not <null>
        at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:152)
        at org.junit.jupiter.api.AssertionFailureBuilder.buildAndThrow(AssertionFailureBuilder.java:132)
        at org.junit.jupiter.api.AssertNotNull.failNull(AssertNotNull.java:49)
        at org.junit.jupiter.api.AssertNotNull.assertNotNull(AssertNotNull.java:35)
        at org.junit.jupiter.api.Assertions.assertNotNull(Assertions.java:312)
        at edu.latece.junit.repo.CourseCatalogTest.testGetCourseNormalizedCase(CourseCatalogTest.java:31)
        at java.base/java.lang.reflect.Method.invoke(Method.java:580)
```

```
Windows PowerShell

[ERROR] edu.latece.junit.repo.CourseCatalogTest.testGetCourseWithWhitespace -- Time elapsed: 0.009 s <<< FAILURE!
org.opentest4j.AssertionFailedError: get() devrait trim les espaces ==> expected: not <null>
        at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:152)
        at org.junit.jupiter.api.AssertionFailureBuilder.buildAndThrow(AssertionFailureBuilder.java:132)
        at org.junit.jupiter.api.AssertNotNull.failNull(AssertNotNull.java:49)
        at org.junit.jupiter.api.AssertNotNull.assertNotNull(AssertNotNull.java:35)
        at org.junit.jupiter.api.Assertions.assertNotNull(Assertions.java:312)
        at edu.latece.junit.repo.CourseCatalogTest.testGetCourseWithWhitespace(CourseCatalogTest.java:45)
        at java.base/java.lang.reflect.Method.invoke(Method.java:580)
        at java.base/java.util.ArrayList.forEach(ArrayList.java:1596)
        at java.base/java.util.ArrayList.forEach(ArrayList.java:1596)

[INFO] Running edu.latece.junit.service.EnrollmentServiceTest
[ERROR] Tests run: 4, Failures: 1, Errors: 1, Skipped: 0, Time elapsed: 0.053 s <<< FAILURE! -- in edu.latece.junit.service.EnrollmentServiceTest
[ERROR] edu.latece.junit.service.EnrollmentServiceTest.testCapacityIncrementAfterValidation -- Time elapsed: 0.015 s <<< FAILURE!
org.opentest4j.AssertionFailedError: Capacité ne devrait pas changer après échec ==> expected: <0> but was: <1>
        at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
        at org.junit.jupiter.api.AssertionFailureBuilder.buildAndThrow(AssertionFailureBuilder.java:132)
        at org.junit.jupiter.api.AssertEquals.failNotEqual(AssertEquals.java:197)
        at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:150)
        at org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:563)
        at edu.latece.junit.service.EnrollmentServiceTest.testCapacityIncrementAfterValidation(EnrollmentServiceTest.java:101)
        at java.base/java.lang.reflect.Method.invoke(Method.java:580)
        at java.base/java.util.ArrayList.forEach(ArrayList.java:1596)
        at java.base/java.util.ArrayList.forEach(ArrayList.java:1596)

[ERROR] edu.latece.junit.service.EnrollmentServiceTest.testTouchingTimeSlotsNoConflict -- Time elapsed: 0.006 s <<< ERROR!
edu.latece.junit.service.EnrollmentException: Enrollment failed: schedule conflict with INF101
        at edu.latece.junit.service.EnrollmentService.enroll(EnrollmentService.java:60)
        at edu.latece.junit.service.EnrollmentServiceTest.testTouchingTimeSlotsNoConflict(EnrollmentServiceTest.java:42)
        at java.base/java.lang.reflect.Method.invoke(Method.java:580)
        at java.base/java.util.ArrayList.forEach(ArrayList.java:1596)
        at java.base/java.util.ArrayList.forEach(ArrayList.java:1596)

[INFO]
[INFO] Results:
[INFO]
[ERROR] Failures:
[ERROR]   TimeSlotTest.testTouchingEndpointsNoOverlap:18 Créneaux adjacents (fin de slot1 = début de slot2) ne devraient PAS se chevaucher ==> expected: <false> but was: <true>
[ERROR]   CourseCatalogTest.testGetCourseNormalizedCase:31 get() devrait normaliser en majuscules ==> expected: not <null>
[ERROR]   CourseCatalogTest.testGetCourseWithWhitespace:45 get() devrait trim les espaces ==> expected: not <null>
[ERROR]   EnrollmentServiceTest.testCapacityIncrementAfterValidation:101 Capacité ne devrait pas changer après échec ==> expected: <0> but was: <1>
[ERROR] Errors:
[ERROR]   EnrollmentServiceTest.testTouchingTimeSlotsNoConflict:42 » Enrollment Enrollment failed: schedule conflict with INF101
[INFO]
[ERROR] Tests run: 11, Failures: 4, Errors: 1, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
```

```
Windows PowerShell
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.028 s
[INFO] Finished at: 2026-03-08T22:27:44-04:00
[INFO] ------------------------------------------------------------------------
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.2.5:test (default-test) on project junit-homework: There are test failures.
[ERROR]
[ERROR] Please refer to C:\Users\assan\Desktop\Workspace\mgl7230\Devoirs\Devoir2\junit-homework2\junit-homework2\target\surefire-reports for the individual test results.
[ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
PS C:\Users\assan\Desktop\Workspace\mgl7230\Devoirs\Devoir2\junit-homework2\junit-homework2>
```

2.2 Résumé des échecs

Erreur détectée :	Description
org.opentest4j.AssertionFailedError: get() devrait normaliser en majuscules ==> expected: not <null>	*La méthode **CourseCatalog.get()** (lignes 21-23) ne normalise pas le code de cours fourni en paramètre. Lorsqu'un cours est enregistré avec le code "INF101" mais recherché avec "inf101" ou "Inf101", la méthode retourne null au lieu de trouver le cours correspondant.*
org.opentest4j.AssertionFailedError: get() devrait trim les espaces ==> expected: not <null>	*Le même problème que précédemment s'applique aux espaces blancs. La méthode **get()** ne supprime pas les espaces avant et après le code de cours **(trim)**. Ainsi, " INF101 " ou "\tINF101\t" ne correspondent pas à "INF101".* ***Impact :** La robustesse de l'application face aux erreurs de saisie utilisateur est compromise.*
org.opentest4j.AssertionFailedError: Capacité ne devrait pas changer après échec ==> expected: <0> but was: <1>	*Dans **EnrollmentService.enroll()** (ligne 46), la capacité du cours est incrémentée AVANT la validation des prérequis et des conflits d'horaire. Le code contient même un commentaire explicite // BUG: incrementer trop tôt. Si l'inscription échoue après cette incrémentation (par exemple, si l'étudiant n'a pas les prérequis), la capacité reste incorrectement modifiée*
edu.latece.junit.service.EnrollmentException: Enrollment failed: schedule conflict with INF101	*Ce test échoue en cascade à cause du Bug #1 dans **TimeSlot.overlapsWith().** L'étudiant tente de s'inscrire à deux cours avec des créneaux adjacents [9:00-10:30) et [10:30-12:00), ce qui devrait être autorisé. Cependant, à cause du bug de chevauchement, le système détecte incorrectement un conflit d'horaire et rejette l'inscription.*

3. Identification des bugs et corrections

3.1 Tableau récapitulatif

Bug #	Classe	Test révélateur	Description & Correction
Bug 1	*TimeSlot*	*testTouchingEndpointsNoOverlap()* testActualOverlap()	***Cause :*** *la méthode de chevauchement utilisait* ***maxStart <= minEnd****, ce qui considère à tort deux créneaux adjacents comme en conflit.* ***Correction :*** *utiliser* ***maxStart < minEnd*** *pour respecter la logique des intervalles* ***[début, fin) (fin exclusive).***
Bug 2	*CourseCatalog*	*testGetCourseNormalizedCase()* *testGetCourseWithWhitespace()*	***Cause : get(courseCode)*** *ne normalise pas l'entrée utilisateur (casse/espaces), donc des cours existants deviennent introuvables avec des variantes de casse.* ***Correction :*** *normaliser avec* ***trim(). toUpperCase()*** *avant l'accès à la map interne.*
Bug 3	*EnrollmentService*	*testCapacityIncrementAfterValidation()*	***Cause*** *: incrementEnrollment() était appelé trop tôt, avant la validation des prérequis et conflits d'horaire, créant des effets de bord (capacité incrémentée même en cas d'échec).* ***Correction :*** *déplacer l'incrément après toutes les validations, juste avant l'enregistrement final.*
Bug 4	*EnrollmentService (via TimeSlot)*	*testTouchingTimeSlotsNoConflict()* *testScheduleConflict()*	***Cause :*** *la validation du conflit d'horaire rejetait à tort les créneaux adjacents qui se touchent mais ne se chevauchent pas (bug indirect via* ***TimeSlot.overlapsWith).*** ***Correction :*** *utiliser uniquement* ***overlapsWith()*** *corrigé qui respecte la logique [début, fin) avec fin exclusive.*

3.2 Détail des corrections

Bug 1 — TimeSlot - Détection de chevauchement d'horaire

Nature du bug: Erreur de logique dans la détection de chevauchement

Code bugué:

```
return maxStart ≤ minEnd;  // INCORRECT
```

Code corrigé

```
return maxStart < minEnd;   // CORRECT
```

Explication:

- Deux créneaux [a,b) et [c,d) se chevauchent uniquement si max(a,c) < min(b,d). La borne de fin étant exclusive, un créneau se terminant à 11h00 et un autre débutant à 11h00 sont donc adjacents, non superposés.
- Avec <=, deux créneaux qui se **touchent** exactement (ex: [10:00,11:00) et [11:00,12:00)) étaient faussement détectés comme se chevauchant
- La borne de fin étant **exclusive**, 11:00 n'appartient pas au premier créneau
- **Impact:** Un étudiant ne pouvait pas s'inscrire à des cours consécutifs (10h-11h puis 11h-12h)

Bug 2 — CourseCatalog - Normalisation des codes de cours

Nature du bug: Recherche incohérente avec l'ajout

Code bugué:

```
public Course get(String courseCode) {
            if (courseCode == null) return null;
            return courses.get(courseCode);  // INCORRECT - pas de
normalisation
}
```

Code corrigé

```
public Course get(String courseCode) {
            if (courseCode == null) return null;
            String normalized = courseCode.trim().toUpperCase();  // CORRECT
            return courses.get(normalized);
}
```

Explication:

- La méthode ***add()*** stocke les cours avec le code normalisé (via ***courseCode*** qui retourne déjà en majuscules)
- La méthode get() ne normalisait pas le paramètre de recherche
- **Résultat:** ***catalog.get("inf1010")*** ou ***catalog.get(" INF1010 ")*** retournait null même si le cours existait
- **Impact:** Les recherches avec des variations de casse ou espaces échouaient systématiquement

Bug 3 — EnrollmentService - Ordre des opérations

Nature du bug: Effet de bord prématuré (violation de l'atomicité)

Code bugué (position initiale):

```
if (!course.hasSeatAvailable()) {
    throw new EnrollmentException("Enrollment failed: capacity reached");
}

course.incrementEnrollment();  // INCORRECT - Trop tôt !

// prereqs
for (String prereq : course.prerequisites()) {
    if (!student.hasCompleted(prereq)) {
        throw new EnrollmentException( ... );
    }
}

// schedule conflict
// ... autres validations ..
```

.

Code corrigé (position finale):

```
if (!course.hasSeatAvailable()) {
    throw new EnrollmentException( ... );
}

// prereqs
for (String prereq : course.prerequisites()) { ... }

// schedule conflict
for (Enrollment e : enrollmentsFor(student.id())) { ... }

// record - APRÈS toutes les validations
course.incrementEnrollment();  // CORRECT - Après validations
Enrollment enrollment = new Enrollment(student.id(), normalized);
enrollments.add(enrollment);
```

Explication:

- ***incrementEnrollment()*** était appelé **après** la vérification de capacité mais **avant** la validation des prérequis et des conflits d'horaire
- Si un étudiant échouait la validation des prérequis ou avait un conflit d'horaire, le compteur était déjà incrémenté

- **Impact:** Le nombre d'inscriptions ne correspondait pas au nombre réel d'étudiants inscrits. Un cours pouvait devenir "plein" alors que des inscriptions avaient échoué
- **Principe violé:** Les modifications d'état doivent être atomiques - tout se fait ou rien ne se fait

4. Réexécution des tests — Code corrigé

Après l'application des corrections décrites à la section 3, la suite de tests a été réexécutée sur le code corrigé. Tous les tests passent désormais avec succès.

```
Windows PowerShell
PS C:\Users\assan\Desktop\Workspace\mgl7230\Devoirs\Devoir2\junit-homework2\junit-homework2> mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] -----------------------< edu.latece:junit-homework >-----------------------
[INFO] Building JUnit Homework Project 1.0.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ junit-homework ---
[INFO] skip non existing resourceDirectory C:\Users\assan\Desktop\Workspace\mgl7230\Devoirs\Devoir2\junit-homework2\junit-homework2\src\main\resources
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ junit-homework ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.4.0:testResources (default-testResources) @ junit-homework ---
[INFO] skip non existing resourceDirectory C:\Users\assan\Desktop\Workspace\mgl7230\Devoirs\Devoir2\junit-homework2\junit-homework2\src\test\resources
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ junit-homework ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ junit-homework ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running edu.latece.junit.domain.TimeSlotTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.058 s -- in edu.latece.junit.domain.TimeSlotTest
[INFO] Running edu.latece.junit.repo.CourseCatalogTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.020 s -- in edu.latece.junit.repo.CourseCatalogTest
[INFO] Running edu.latece.junit.service.EnrollmentServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.022 s -- in edu.latece.junit.service.EnrollmentServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.596 s
[INFO] Finished at: 2026-03-08T22:52:19-04:00
[INFO] ------------------------------------------------------------------------
PS C:\Users\assan\Desktop\Workspace\mgl7230\Devoirs\Devoir2\junit-homework2\junit-homework2>
```